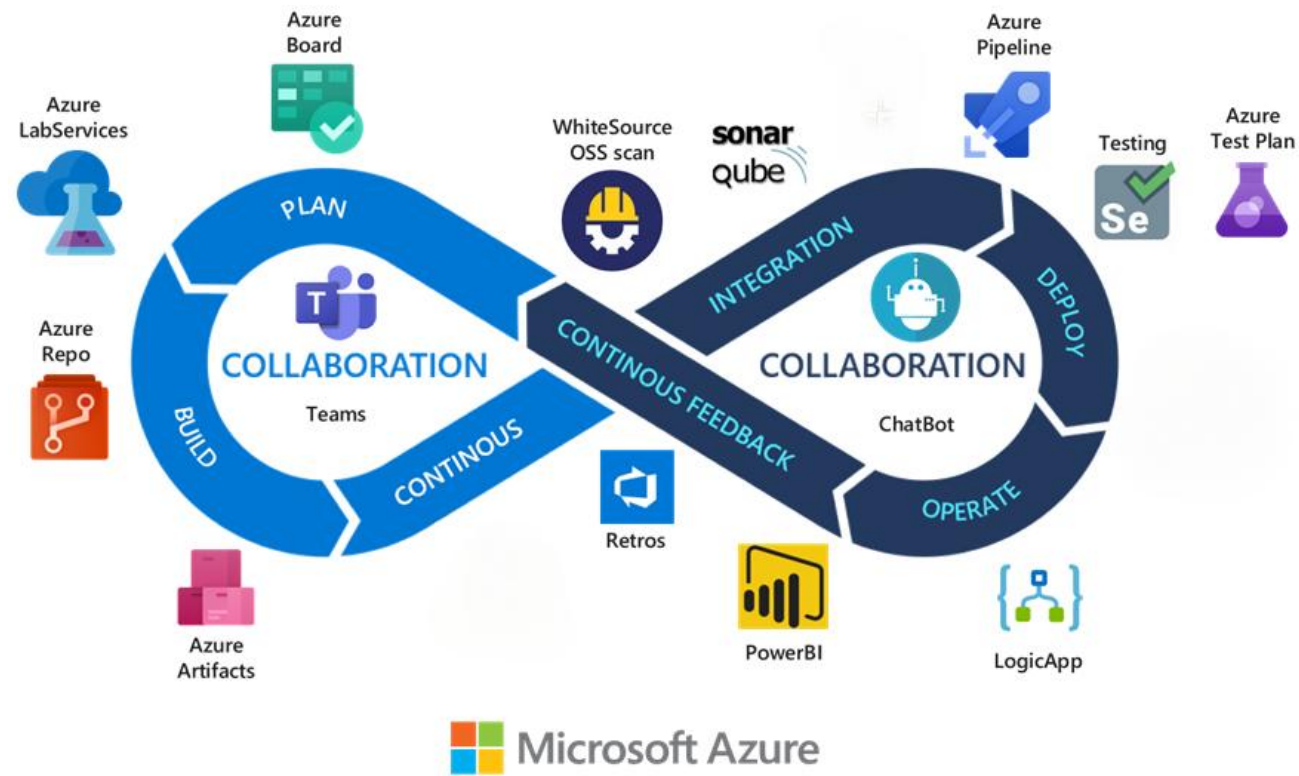




CI/CD Internal Service Proposal

โครงสร้างระบบ CI/CD สำหรับ Implement งานโครงการต่าง ๆ



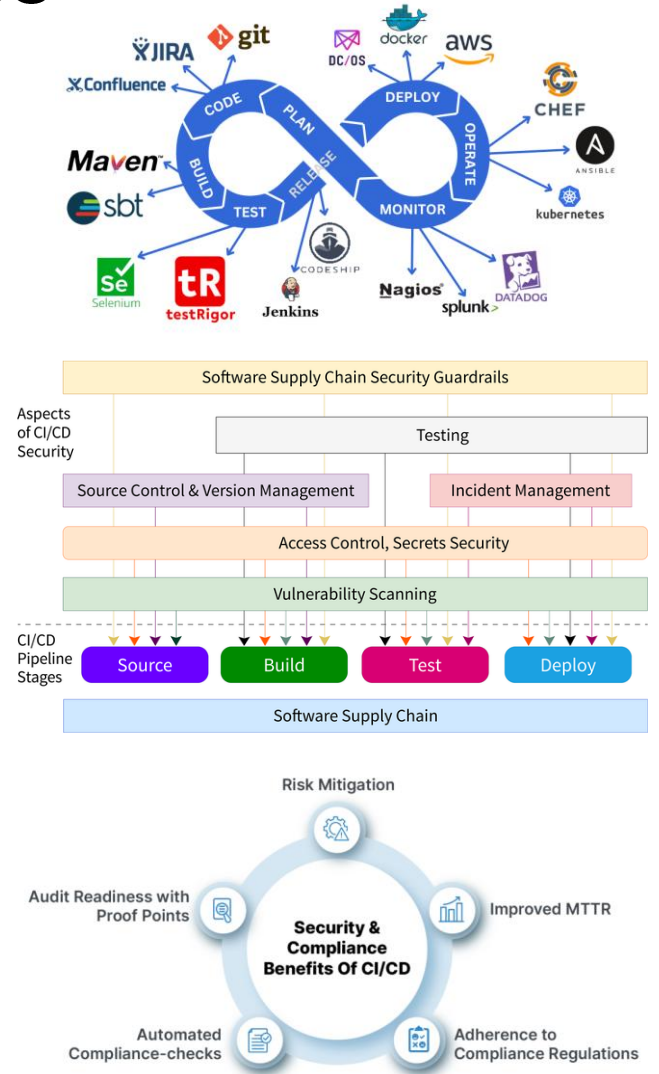
Agenda

- 1) ประเด็นปัญหาถึงความต้องการระบบ CI/CD Service
- 2) โครงสร้างระบบ CI/CD Services ที่เหมาะสม
- 3) องค์ประกอบและขั้นตอนการทำงานของ CI/CD Services
- 4) เจ็อนไข ราคาและการติดตั้งการใช้งาน
- 5) Output ของ CI/CD Services ที่คาดหวัง

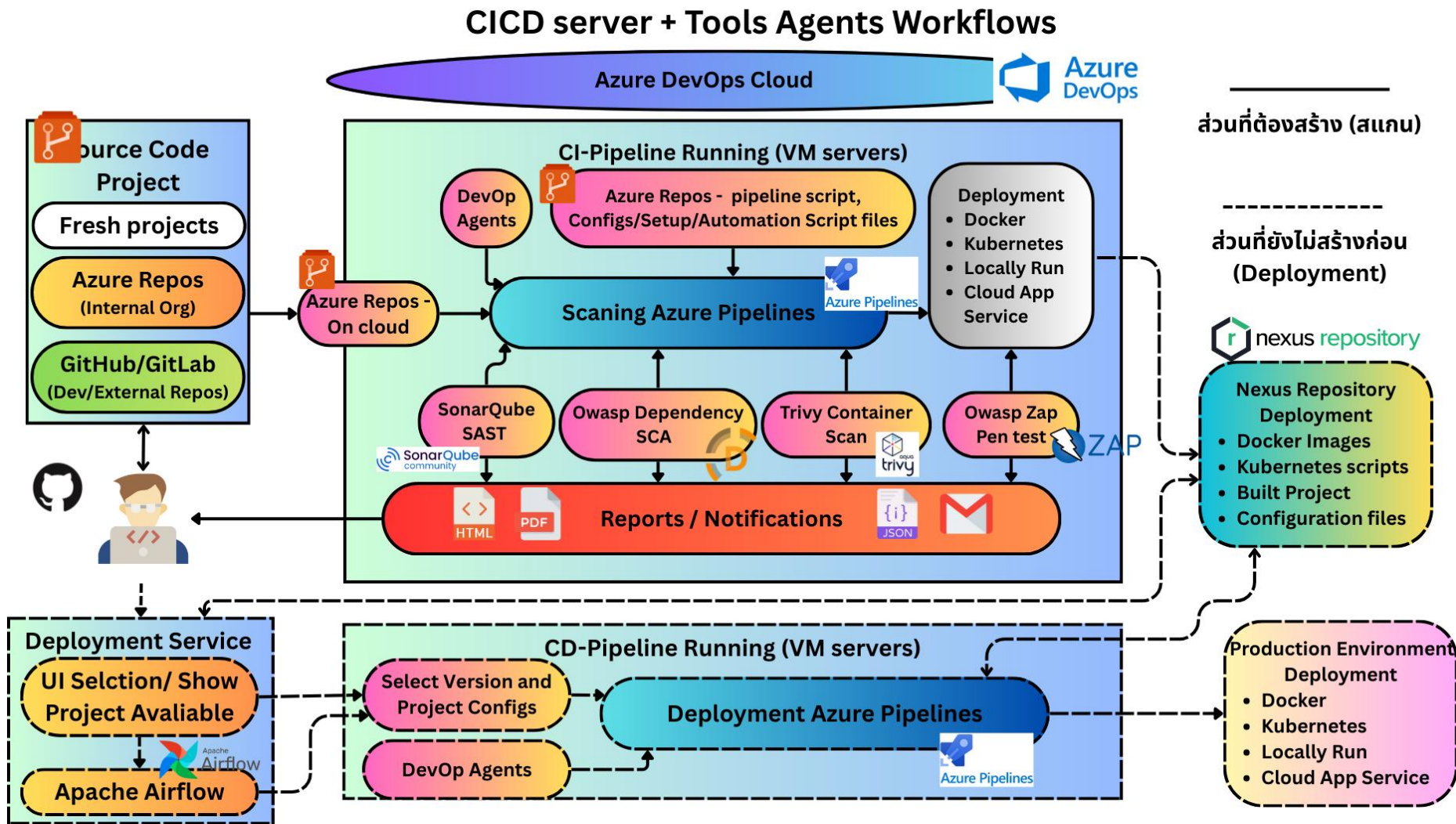
ประเด็นปัญหาถึงความต้องการระบบ CI/CD Service

กระบวนการแบบ CI/CD มีประเด็นหลัก ๆ ดังนี้

- 1) พัฒนาแบบต่อเนื่องและสามารถตรวจสอบการใช้งาน: จากเดิมส่วนใหญ่เน้น **การทำด้วยมือผ่านทีม Dev และ Tester ที่ควบคุมยากและแยกทดสอบที่ไม่ต่อเนื่อง** เปลี่ยนเป็น **ระบบอัตโนมัติ (Automation) ที่ตรวจสอบได้ทันทีและตรวจสอบย้อนหลัง** ได้ผ่านชุด unit tests
- 2) ตรวจสอบและทดสอบในมาตรฐานความปลอดภัย: จาก **ภัยคุกคามไซเบอร์ที่รุนแรงและการตรวจสอบช่วงท้ายโครงการที่ล่าช้า** จึงจำเป็นต้องมี **ระบบรับการตรวจสอบความปลอดภัยทุกครั้งที่มีการแก้ไข Code** เพื่อให้ทีมพัฒนาเห็นช่องโหว่และแก้ไขได้ทันทีโดยไม่ต้องรื้อระบบใหม่ภายหลัง
- 3) รองรับการถูกโจมตีและกู้ระบบโครงการเดิม: จากเดิมที่ **ไฟล์กระจายและยากต่อการบูรณาการ** จำเป็นที่ต้องมี **CI/CD Platform ให้เกิดจัดเก็บไฟล์พร้อมใช้ (Artifacts) แบบรวมศูนย์** ที่เชื่อมโยงทุกส่วนประกอบไว้ในที่เดียว พร้อมสำหรับการดึงข้อมูลกลับมาติดตั้งใหม่ (Restore) ได้ทันทีเมื่อเกิดเหตุฉุกเฉิน
- 4) เสริมความน่าเชื่อถือพร้อมรองรับการปฏิบัติตามมาตรฐานสากล: จาก **เดิมที่ขาดหลักฐานยืนยันคุณภาพที่เป็นรูปธรรม** ในระบบ CI/CD ควรมี **ระบบออกรายงานผลอัตโนมัติ (Auto-Reports) ที่ชัดเจน** เพื่อใช้ยืนยันมาตรฐานความปลอดภัย (Compliance) ต่อผู้ตรวจสอบหรือลูกค้าได้ทันที



โครงสร้างระบบ CI/CD Services ที่เหมาะสม



- เลือกใช้ **Azure DevOps แบบ Cloud** เป็นศูนย์กลางสิ่งการ เนื่องจากเริ่มต้นใช้งานได้ฟรี (สำหรับทีมขนาดเล็ก 5 users แรก) และไม่ต้องเสียค่าบำรุงรักษา Server สำหรับตัว Controller
- ใช้ **VM Servers ผ่าน Azure benefits 1** ตัว รันเครื่อง VM 1 ตัวขนาด CPU 4 core, RAM 16 GB ซึ่งเพียงพอกับเครื่องมือ **Scanner** ทั้งหมดและ **Logs Storage** และเดิมมี Microsoft-hosted agents ให้ 1,800 minutes free
- เหมาะกับใช้ในองค์กรขนาดเล็ก หรือกลางที่โครงการไม่มาก



องค์ประกอบและขั้นตอนการทำงานของ CI/CD Services

1. องค์ประกอบและเครื่องมือ (Services & Tools Breakdown)

A. ส่วนจัดการ Source Code (Source Code Management)

ของเดิมในองค์กรที่มีอยู่

- **Azure Repos (Internal Org):** ใช้เก็บ Source Code หลักของโครงการภายในองค์กร - เป็น Azure Repos Enterprise
- **GitHub / GitLab (Dev/External Repos):** เชื่อมต่อกับ Source Code จากภายนอก หรือ Open-Source Projects ของใหม่
- **Azure Repos (On Cloud):** จุดพัก Code บน Cloud ที่ Sync มาจากแหล่งต่าง ๆ เพื่อเตรียมส่งเข้าสู่ Pipeline

B. ส่วนควบคุมและสั่งการ (Control Plane)

- **Azure DevOps Cloud:** เป็นสมองกลางของระบบ ทำหน้าที่จัดการ Project, User และ Orchestrate (สั่งการ) Pipeline ทั้งหมด

* Azure DevOps Cloud service มีประเด็นที่ data ใน Azure Repos และ logs การรัน service จะถูกเก็บที่ ฐานข้อมูล US ไม่ใช่ Local หากมีเรื่อง Data Privacy เป็นเรื่องที่ต้องคำนึง จำเป็นต้องใช้ เครื่องมืออื่น ๆ ที่ใกล้เคียง เช่น Jenkin, Argo Workflow แทน



องค์ประกอบและขั้นตอนการทำงานของ CI/CD Services

1. องค์ประกอบและเครื่องมือ (Services & Tools Breakdown)

C. ส่วนตรวจสอบความปลอดภัย (CI-Pipeline & Security Tools) ทำงานบน **VM Servers** (เครื่องเซิร์ฟเวอร์ภายใน) โดยใช้ **DevOps**

Agents เป็นตัวรันงาน ประกอบด้วยเครื่องมือตรวจสอบความปลอดภัย 4 ด้าน:

- **SonarQube (SAST):** สแกน Source Code เพื่อหาบัก (Bugs), กลิ่นของโค้ดที่ไม่ดี (Code Smells), และช่องโหว่ทางภาษาโปรแกรม
- **OWASP Dependency Check (SCA):** สแกน Library และ 3rd-party packages ที่โปรเจกต์ดึงมาใช้งาน ว่ามีประวัติช่องโหว่ (CVE) หรือไม่
- **Trivy (Container Scan):** สแกน Docker Images เพื่อหาช่องโหว่ในระดับ OS Layer และ Application Packages
- **OWASP ZAP (DAST/Pen Test):** ทำการจำลองยิงทดสอบเจาะระบบ (Penetration Testing) ไปยังแอปพลิเคชันเบื้องต้นเพื่อดูการตอบสนองจริง
- **Reports / Notifications:** ระบบแจ้งเตือนผลลัพธ์ผ่าน Email ในรูปแบบ HTML, PDF, หรือ JSON



องค์ประกอบและขั้นตอนการทำงานของ CI/CD Services

1. องค์ประกอบและเครื่องมือ (Services & Tools Breakdown)

D. ส่วนจัดเก็บไฟล์ (Artifact Management)

- **Nexus Repository:** เป็นคลังกลางสำหรับจัดเก็บไฟล์ที่ผ่านการ Build และ Scan แล้ว (Artifacts) เช่น Docker Images, Scripts, และ Config Files เพื่อเตรียมพร้อมสำหรับการ Deploy

E. ส่วนจัดการการติดตั้ง (Deployment Orchestration)

- **Apache Airflow:** ทำหน้าที่เป็น Workflow Management ให้ผู้ดูแลระบบเลือกเวอร์ชัน (UI Selection) และสั่งการ Deploy ได้อย่างเป็นระบบ
- **Deployment Service:** หน้าจอ UI สำหรับดูโปรเจกต์ที่พร้อมใช้งานและเลือก Config

F. ส่วนปลายทาง (Production Environment) รองรับการ Deploy ไปยังสภาพแวดล้อมที่หลากหลาย ได้แก่ **Docker, Kubernetes, Local Run (On-premise), และ Cloud App Service**



องค์ประกอบและขั้นตอนการทำงานของ CI/CD Services

2. ขั้นตอนการทำงาน (Workflow Description)

1. Step 1: Code Ingestion (นำเข้าโค้ด)

- Developer ทำการ Push Code ขึ้นสู่ **Azure Repos (Internal)** หรือ **GitHub/GitLab**
- ระบบจะทำการ Sync Code เข้าสู่ **Azure Repos On Cloud** เพื่อเตรียมเข้าสู่กระบวนการ CI

2. Step 2: Continuous Integration & Security Scanning (ตรวจสอบความปลอดภัย)

- **Trigger:** เมื่อมี Code ใหม่เข้ามา **Azure DevOps Cloud** จะสั่งให้ **DevOps Agents** บน VM Server เริ่มทำงาน
- **Execution:** Agent จะดึง Pipeline Script และ Code มาทำการ Build และรันเครื่องมือ Security ทั้ง 4 ตัว (SonarQube, OWASP Dependency, Trivy, OWASP ZAP) เรียงตามลำดับ
- **Reporting:** หากพบช่องโหว่ร้ายแรง ระบบอาจหยุดการทำงาน (Fail) และส่ง Report แจ้งเตือนกลับไปทาง Email



องค์ประกอบและขั้นตอนการทำงานของ CI/CD Services

2. ขั้นตอนการทำงาน (Workflow Description)

3. Step 3: Artifact Storing (จัดเก็บชิ้นงาน)

- เมื่อผ่านการตรวจสอบทุกขั้นตอน ไฟล์ที่ได้ (เช่น Docker Image หรือ Built Project) จะถูก Push ไปเก็บรักษาไว้ที่ **Nexus Repository** เพื่อเป็น "Golden Image" ที่ปลอดภัยและห้ามแก้ไข

4. Step 4: Deployment Triggering (สั่งการติดตั้ง)

- เมื่อต้องการนำขึ้นใช้งานจริง ผู้ดูแลระบบจะเข้าผ่าน **Deployment Service (UI)** ที่เชื่อมต่อกับ **Apache Airflow**
- ทำการเลือกเวอร์ชันของซอฟต์แวร์จาก Nexus และเลือก Environment ปลายทาง

5. Step 5: Continuous Delivery (ส่งมอบระบบ)

- Apache Airflow จะส่งคำสั่งไปยัง **CD-Pipeline** (บน VM Servers)
- **DevOps Agents** ในส่วน CD จะดึง Artifacts จาก Nexus และนำไปติดตั้ง (Deploy) บน Production Environment (เช่น Kubernetes Cluster หรือ Cloud App) ตามที่กำหนดไว้



เงื่อนไข ราคาและการติดตั้งการใช้งาน

Azure Pipelines

1 Free Microsoft-hosted CI/CD
1 Free Self-Hosted CI/CD

Start free

- 1 Microsoft-hosted job with 1,800 minutes per month for CI/CD and 1 self-hosted job with unlimited minutes per month
- \$40 per extra Microsoft-hosted CI/CD parallel job and \$15 per extra self-hosted CI/CD parallel job with unlimited minutes

Microsoft-hosted
Free

1,800 minutes free with 1 free parallel job

\$0/mo

Self-hosted
1

1 free parallel job with unlimited minutes

\$0/mo

Azure Artifacts

2 GiB free, then starting at \$2 per GiB

Start free

- Industry-leading NuGet Server
- Support for Maven, npm, and Python packages
- Upstream sources to help protect open-source dependencies
- Integrated with Azure Pipelines
- Sophisticated access controls

First 2 GiB free

0

\$0/extra gigabyte

Rate card

- 0 - 2 GiB = Free
- 2 - 10 GiB = \$2 per GiB
- 10 - 100 GiB = \$1 per GiB
- 100 - 1,000 GiB = \$0.50 per GiB
- 1,000+ GiB = \$0.25 per GiB

GitHub Advanced Security for Azure DevOps

Code Security: \$30 per committer per month

Activate now

- Code scanning
- Dependency scanning

Secret Protection: \$19 per committer per month

Activate now

- Secret scanning with push protection

Code Security committers
0

\$30/committer/month

Secret Protection committers
0

\$19/committer/month

\$0/mo

Basic Plan

First 5 users free, then \$6 per user per month

Start free

- Azure Pipelines:** Includes the free offer from INDIVIDUAL SERVICES
- Azure Boards:** Work item tracking and Kanban boards
- Azure Repos:** Unlimited private Git repos
- Azure Artifacts:** 2 GiB free per organization

First 5 users free. For additional users, use slider

0

Additional users are charged \$6/user/month

\$0/mo

Basic + Test Plans

\$52 per user per month

30 day free trial

- Includes all Basic plan features
- Test planning, tracking & execution
- Browser-based tests with annotation
- Rich-client test execution
- User acceptance testing
- Centralized reporting

\$52 per user per month

1

\$52/user/month

\$52/mo

- Azure DevOps Cloud Service** มีโครงสร้างราคาที่ยืดหยุ่นตามการใช้งานจริง โดยฟรี 5 users แรก และค่าบริการเริ่มต้นที่ \$6 ต่อ user ต่อเดือน
- Azure Repos:** Git repositories ไม่จำกัดจำนวนแบบ Private
- Azure Pipelines:** 1 Microsoft-hosted CI/CD job ฟรี (1,800 นาที/เดือน) และ 1 Self-hosted job ฟรี (ติดตั้ง Self-hosted Agents ได้หลาย ๆ ตัวแต่ 1 ตัวรันทีละ job ต่อรอบไม่มี parallel) หรือ เมื่อต้องการ parallel jobs เพิ่ม:
 - Microsoft-hosted: \$40/job/month สำหรับ minutes ไม่จำกัด
 - Self-hosted: \$15/job/month สำหรับ minutes ไม่จำกัด
- Azure Artifacts:** พื้นที่จัดเก็บ package 2 GiB ฟรี จากนั้น \$2 ต่อ GiB

เจ็อนไซ ราคาและการติดตั้งการใช้งาน



VM 1 *(Dev Stage)*
Scanner server

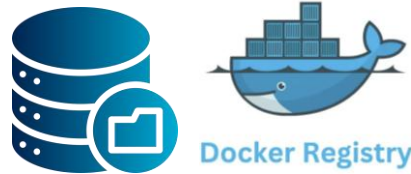








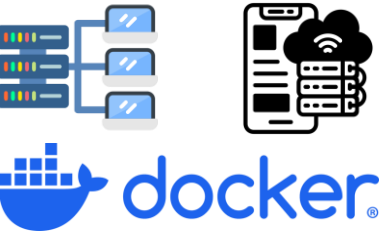
VM 2 *(Deploy Stage)*
Repository server

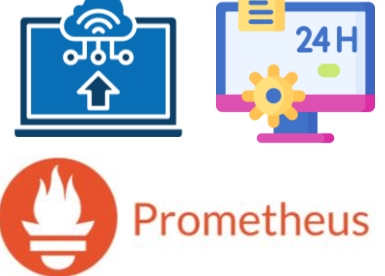






VM 3 *(Deploy Stage)*
Deployment server






ชื่อเครื่อง (Role)	OS	จำนวน	รุ่น Azure VM แนะนำ (Size)	สเปค (Specs)	เหตุผล
VM 1-A (Linux Agent)	Ubuntu	1	Standard_D4s_v5	4 vCPU, 16GB RAM OS: 32 GB Data: 128 GB (Premium SSD)	รัน Docker กับ Scan tools ตรวจสอบงานส่วนใหญ่ ต้องมีการใช้หลายเครื่องมือพร้อมกัน
VM 1-B (Win Agent)	Windows	1	Standard_D4s_v5	4 vCPU, 16GB RAM OS: 128 GB Data: 128 GB (Premium SSD)	สำหรับงาน .NET Framework หรือ Legacy App ที่รันบน Windows เท่านั้น
VM 2 (Repo)	Ubuntu	1	Standard_D2s_v5	2 vCPU, 8GB RAM OS: 32 GB Data: 512 GB (Standard SSD)	เน้นเสถียรภาพ + เพิ่ม Data Disk (SSD) ตามขนาดข้อมูล
VM 3 (Deploy)	Ubuntu	1	Standard_B2s_v2	2 vCPU, 4GB RAM OS: 32 GB Data: 64 GB (Standard SSD)	งานเบา ใช้รุ่น Burstable (B-series) ประหยัดงบได้



เงื่อนไข ราคาและการติดตั้งการใช้งาน

กรณีเป็น **Deployment Service** จะทำหน้าที่ในช่วงท้ายของ Cycle (หรือเมื่อจบ Sprint) เพื่อนำซอฟต์แวร์ที่ผ่านการทดสอบแล้ว (Artifacts) ไปติดตั้งยังสภาพแวดล้อมจริง (Production) โดยระบบต้องรองรับการ Deploy ไปยังหลายปลายทาง (Multi-target Deployment) ทั้ง Server ภายในองค์กร (On-Premise), Cloud Server หรือ Kubernetes Cluster ของลูกค้า

โดยคร่าว ๆ **Deployment Service** มีกระบวนการทำงานดังนี้:

1. **Selection & Orchestration (การเลือกและสั่งการ):** ผู้ดูแลระบบหรือ Project Manager จะเข้าใช้งานผ่านหน้าเว็บ (UI Selection) ที่สร้างด้วย **Apache Airflow** เพื่อเลือก "ชื่อโปรเจกต์" และ "เวอร์ชันของซอฟต์แวร์" (Version/Tag) ที่ต้องการติดตั้ง
2. **Artifact Retrieval (การดึงไฟล์ติดตั้ง):** ระบบจะไปดึงไฟล์ Image หรือ Package ที่ระบุจาก **Nexus Repository (VM 2)** โดยอัตโนมัติ เพื่อรับประกันว่าสิ่งที่นำมาติดตั้งคือตัวเดียวกับที่ผ่านการ Scan และ Test มาแล้ว (Immutability)
3. **Configuration Injection (การใส่ค่าตั้งต้น):** ระบบจะทำการแทนที่ค่า Config ต่าง ๆ ให้เหมาะสมกับสภาพแวดล้อมปลายทาง (เช่น Database Connection ของ Production vs UAT) ผ่าน Script อัตโนมัติ
4. **Execution (การติดตั้ง):** Azure DevOps Agent บน VM 3 จะยิงคำสั่งติดตั้ง (Deploy Command) ไปยังเครื่องเป้าหมาย (Docker, Kubernetes, หรือ Web Server)
5. **Health Check & Monitoring (การตรวจสอบหลังติดตั้ง):** เมื่อติดตั้งเสร็จสิ้น ระบบ Monitoring (Prometheus/Grafana) จะทำการตรวจสอบสถานะกันว่า Service นั้น Up & Running หรือไม่



Output ของ CI/CD Services ที่คาดหวัง